Detecting Negative Company News with a Fine-Tuned Transformer

Mauro Reverberi

Udacity Institute of AI & Technology / Woolf

Master of Science in AI, Capstone Project 4: Deep Learning Systems

October 9, 2026

Detecting Negative Company News with a Fine-Tuned Transformer

Report Overview

In this project I fine-tuned a pretrained Transformer in PyTorch that decides whether a sentence of financial news about a company is negative, neutral or positive for an investor. The data is 4,836 labeled sentences from the Financial PhraseBank, and the model is the DistilBERT encoder with my own classification head. As the controlled comparison I trained the same model with a loss that gives the rare negative class more weight, both configurations with five seeds. The model is a first step towards adverse media screening for a later due diligence agent in my capstone series, although the labels measure investor sentiment and not compliance relevance.

Dataset and Task Description

The Financial PhraseBank (Malo et al., 2014) holds sentences from English financial news about companies, many of them Finnish. Five to eight of 16 annotators with a background in finance labeled every sentence as negative, neutral or positive from an investor's perspective. The release has four files with more than 50%, 66%, 75% and full agreement. I downloaded version 1.0 from https://huggingface.co/datasets/takala/financial_phrasebank. The sentences are real news text, the license (CC BY-NC-SA 3.0) allows academic use, and I did not use the source in my earlier capstone projects.

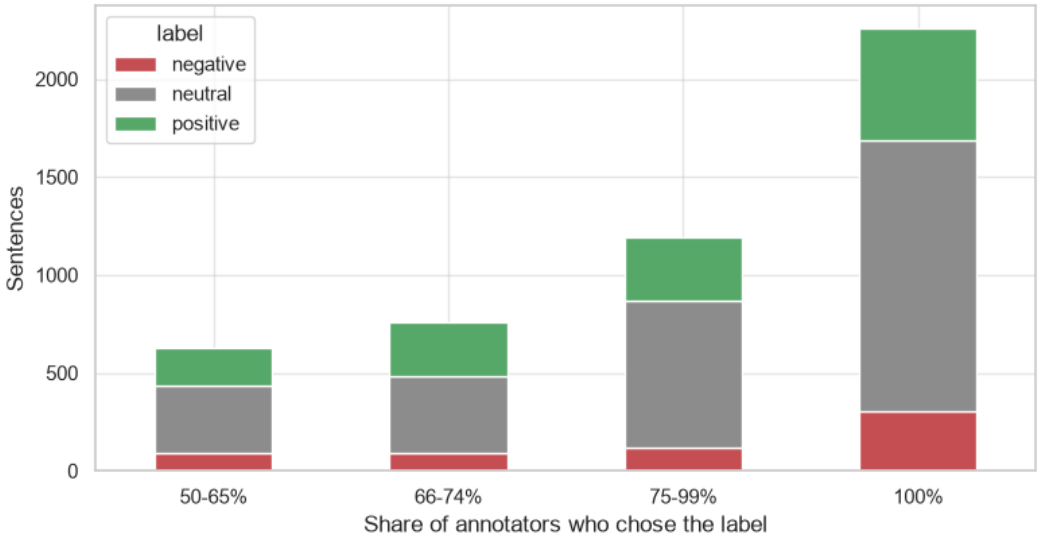
My script prepare_dataset.py downloads the archive from a fixed revision and checks its SHA-256. It removes 4 records with conflicting labels and 6 repeated records, which leaves 4,836 of the 4,846 lines, and it gives every sentence the level of the strictest file that contains it (50-65%, 66-74%, 75-99% or 100%). Many sentences are the same template with other amounts or years. If such variants sit in training and test, the test partly measures what the model has memorized, and the results look better than they are (Elangovan et al., 2021). The script

therefore puts sentences into one group if they are equal once everything except the letters is removed, or if their TF-IDF cosine similarity is at least 0.65. 123 of these groups hold more than one sentence, 338 sentences in total, and each remaining sentence is a group of its own. The groups are split about 70/15/15 into 3,393 training, 721 validation and 722 test sentences, stratified by the label and agreement level of the first sentence of each group, with seed 42.

The labels are imbalanced. 59.4% of the sentences are neutral, 28.1% positive and only 12.5% negative, with the same shares in all three splits within about one percentage point. In Figure 1 the largest bar is the one with full agreement, 2,259 sentences or 46.7%. Among the sentences with less than 75% agreement 34.1% are positive, against 25.2% among those with full agreement.

Figure 1

Sentences per Agreement Level and Label



The task needs the relation between the parts of a sentence. A number appears in 75.3% of the negative sentences but only in 46.7% of the neutral ones, and the sentiment can depend on how two figures compare, for example a return on investment of 4.1% against 43.8% in the first half of 2008. With a median of 28 tokens and a maximum of 150, no sentence reaches my limit

of 160 tokens. The tokenizer lower-cases the text and splits it into the word pieces of the BERT vocabulary (Devlin et al., 2019). 93 sentences (1.9%) contain one recurring form of broken letters, for example Sepp+ñl+ñ for Seppälä, and other encoding errors such as -¦ for an apostrophe remain as well. I kept all of them unchanged, because some patterns cannot be repaired without guessing.

Model Architecture and Design Decisions

The baseline is the pretrained encoder distilbert-base-uncased with a small classification head. DistilBERT has six instead of the twelve Transformer blocks of BERT, was trained to copy the larger model and keeps about 97% of its performance on the GLUE benchmark (Sanh et al., 2019). With only 3,393 training sentences I chose a pretrained encoder, because it brings language knowledge from a much larger corpus, and Devlin et al. (2019) fine-tune BERT on tasks with only about 2,500 training examples. An LSTM trained from scratch would have to learn the language from these few sentences. It also reads word by word, so in "net loss widened to EUR 189,000 ... from EUR 10,000" the two amounts meet only through its hidden state (Goodfellow et al., 2016). Self-attention lets every token use all other tokens of the sentence directly (Vaswani et al., 2017). I did not train an LSTM for comparison, so this choice rests on these arguments and not on my own numbers.

The head takes the vector of the [CLS] token, which Devlin et al. (2019) use as the summary of the input for classification, through a linear layer with 768 units, then ReLU, dropout 0.1 and a second linear layer that gives three logits. This is the structure of the DistilBERT classification head in the transformers library (Hugging Face, n.d.-a), only with the dropout 0.1 of the encoder instead of its default 0.2, and I kept it as it is. ReLU is the common default for hidden layers, three logits with a softmax fit a task with exactly one of three labels,

and dropout switches off random units during training to reduce overfitting (Goodfellow et al., 2016). The model has 67.0 million parameters, all of them trainable, so encoder and head are fine-tuned end to end. The transformers library provides the DistilBERT implementation with its six blocks and 12 attention heads, and its weights. The head, the training loop and the evaluation are my own PyTorch code.

The loss is cross-entropy, the standard loss for single-label classification (Goodfellow et al., 2016). The optimizer is AdamW, Adam with decoupled weight decay (Loshchilov & Hutter, 2019), with a learning rate of 2e-5, weight decay 0.01 and batch size 32. Devlin et al. (2019) fine-tuned BERT with batch sizes of 16 and 32, learning rates from 2e-5 to 5e-5 and two to four epochs. I took the low learning rate because the pretrained weights only need to be adapted, and five epochs, one more than their upper end, so the curves show when the model starts to overfit. I left out warm-up and learning rate decay to keep the training loop simple. After every epoch I keep the weights with the best validation macro-F1, the selection idea of early stopping (Goodfellow et al., 2016). I select on macro-F1 and not on the validation loss, because all three classes should count equally in the comparison.

One trained model does not show how much the result depends on the random parts of the training, so I trained each configuration with the same five seeds, 42 to 46. A seed fixes the head initialization, the dropout and the batch order. As floors I report a model that always answers neutral and a logistic regression on TF-IDF features of words and word pairs, once with equal and once with balanced class weights. TF-IDF weights a word by how often it appears in a sentence and gives words that appear in many sentences less weight.

My results show that this setup works for my task, but they do not test each design choice separately. The Transformer reaches a mean test macro-F1 of 0.8176, about 0.12 more than the

balanced TF-IDF model (0.6928), although this comparison does not isolate what the pretraining adds. One model trains in about six minutes on my CPU, so the ten runs took about one hour. In the main run the validation loss rises after epoch 2 or 3 (Figure 2), and in eight of the ten runs the last epoch was not the best one, which supports keeping the weights of the best epoch.

Experimental Comparison

The experiment changes one thing, the loss function, which is a change of the training strategy. In the second configuration the cross-entropy gives every class a weight, the number of training sentences divided by three times the number of sentences of that class. This gives 2.643 for negative, 0.561 for neutral and 1.192 for positive, so for the same unweighted loss a negative sentence gets almost five times the weight of a neutral one. Everything else stays the same, the encoder, the head, the data, the optimizer settings, the seeds, the batch order and the best-epoch rule.

Weighting the classes by their inverse frequency is a standard answer to imbalanced data, because otherwise the rare class has little influence on the loss (He & Garcia, 2009). Araci (2019) also weights the cross-entropy loss for FinBERT on the same data, with the milder square root of the inverse frequency. For the screening tool I have in mind I assume that a missed negative item costs more than an extra item to review. So the question is how many more negative sentences the weighted loss finds and how much precision this costs.

Results and Interpretation

Figure 2 shows the loss curves of the main run with seed 42. In five epochs both training losses drop from 0.65 and 0.68 to below 0.1, while the validation losses turn upwards early. For the baseline the lowest validation loss is 0.3805 after epoch 3, and it ends at 0.4988. For the weighted model the minimum is 0.3965 after epoch 2, and it ends at 0.4845. A training loss that

keeps falling while the validation loss rises is a sign of overfitting (Goodfellow et al., 2016). Because the two training losses use different class weights, I compare the configurations on the same unweighted validation loss. Figure 3 adds the validation macro-F1 of all ten runs. It does not follow the loss and stays between about 0.79 and 0.86 from epoch 2 on.

Figure 2

Training and Validation Loss per Epoch for Both Configurations, Seed 42

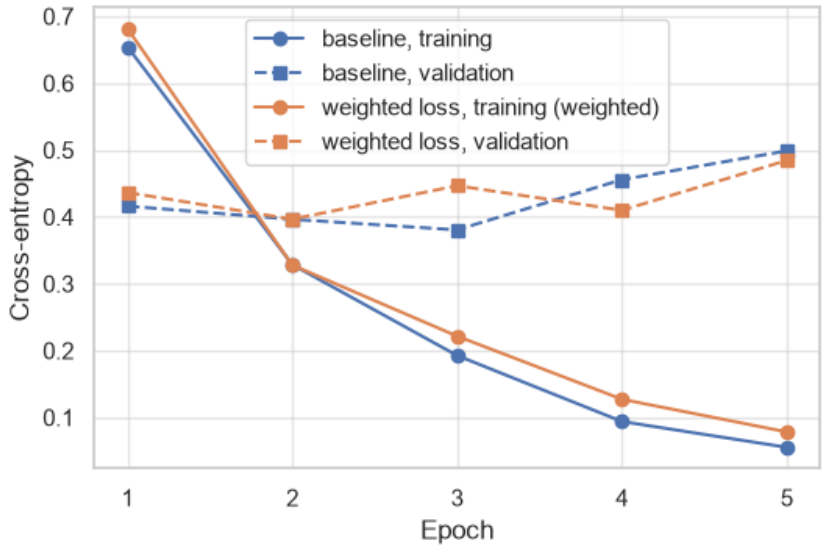
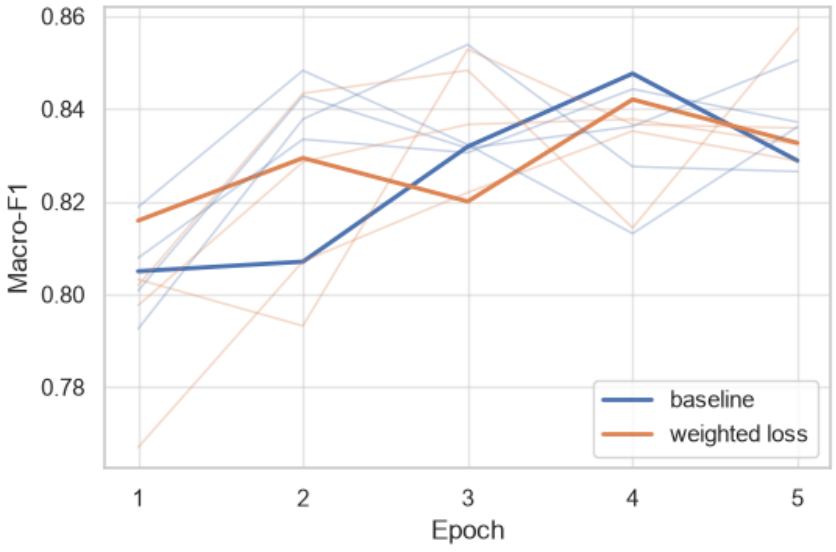


Figure 3

Validation Macro-F1 per Epoch for Both Configurations and All Seeds



Before the first test result I fixed macro-F1, the mean F1 of the three classes, as the main metric, because accuracy is dominated by the majority class (He & Garcia, 2009). The recall of a class is the share of its sentences that the model finds, the precision is the share of correct flags, and He and Garcia (2009) list both among the metrics for imbalanced data. Both configurations are far above the floors, the constant model with macro-F1 0.2470 and the TF-IDF model with 0.6483, or 0.6928 with balanced class weights. Averaged over the five seeds the baseline reaches a test macro-F1 of 0.8176 with a standard deviation of 0.0043, and the weighted model reaches 0.8191 with 0.0023. On the negative class the weighted loss moves the mean recall up from 0.8483 to 0.8644 and the mean precision down from 0.7386 to 0.7294. All three changes are smaller than the spread between the seeds of one configuration.

Figure 4 holds the confusion matrices of seed 42. Of the 87 negative test sentences the baseline finds 73 and the weighted model 77, and the price is 32 false alarms instead of 25. The border between neutral and positive moves as well. The weighted model calls fewer positive sentences neutral, 31 instead of 39, but more neutral sentences positive, 44 instead of 36, and ends with 117 errors against 114. Among the five seeds, seed 42 and seed 43 have the largest recall gain, and seed 42 is one of the two seeds where the weighted model has the lower macro-F1. So Figure 4 shows the trade-off clearer than most other seeds would.

Figure 4

Confusion Matrices of Both Configurations on the Test Split, Seed 42

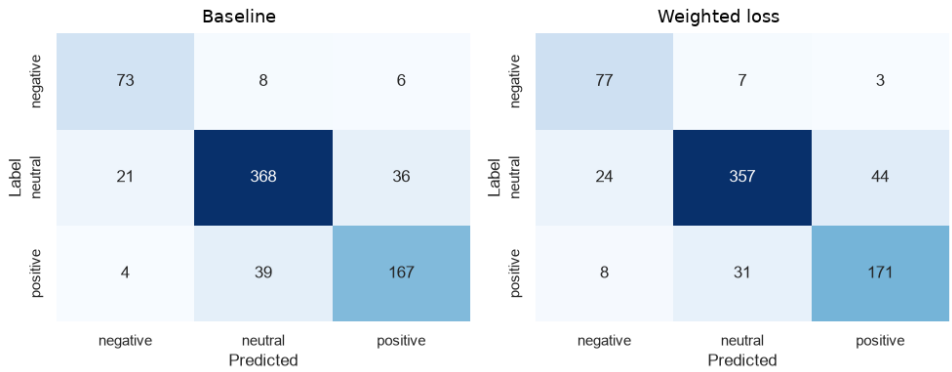
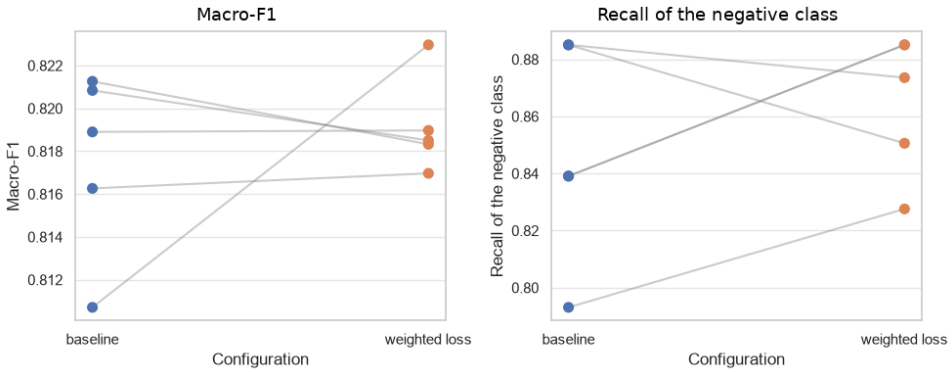


Figure 5 compares the two configurations seed by seed. The weighted model finds 3 or 4 more negative sentences in three seeds and 1 and 3 fewer in the other two, and it has the higher macro-F1 in 3 of 5 seeds. In the recall panel the lines for seeds 42 and 43 lie on top of each other. Over the five seeds I used two tests that Demšar (2006) describes for comparing two classifiers over several data sets. The paired t-test checks whether the mean difference is zero and assumes roughly normal differences, which five values hardly confirm. The sign test only counts in how many seeds the weighted model wins, so it needs no normal distribution, but with five pairs it cannot fall below $p = 0.0625$. The two-sided t-tests give $p = 0.605$ for macro-F1 and $p = 0.385$ for the negative recall, and the sign test gives $p = 1.0$ for both, so neither difference is significant at the 5% level. These tests describe the training variation on this fixed split and do not measure the uncertainty over new training or test samples. With only five differences the p-values are a rough check, and they do not show that the configurations are equivalent. Counted in sentences, the weighted model finds 1.4 more negative sentences per seed and raises 1.6 more false alarms, so I read the result as a small shift towards the negative class, and neither configuration is clearly better.

Figure 5

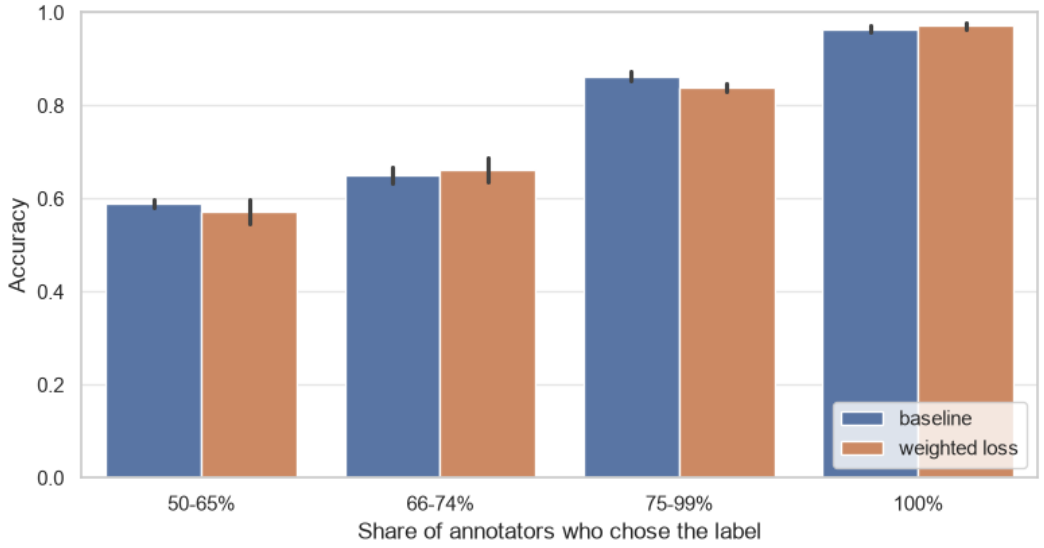
Test Macro-F1 and Recall of the Negative Class per Seed



The clearest pattern in my results is the agreement of the annotators (Figure 6). Where all annotators agreed, 335 test sentences, the baseline is right on 96.4% of them on average. Where only 50-65% agreed, 96 sentences, it is right on 59.0%, and the accuracy drops with every step in between, for both configurations. The agreement levels also differ in their mix of labels, so this pattern does not prove that unclear sentences alone cause the errors. Part of these errors may be a different reading of an unclear sentence, although my results cannot tell which ones.

Figure 6

Test Accuracy per Agreement Level, Mean and Standard Deviation over the Five Seeds



Among the eight errors in which the baseline is most confident, seven are positive sentences that it calls neutral with an uncalibrated confidence between 0.995 and 0.998, and five of them have only 50-65% agreement. Several are plans or contracts, for example Etteplan's target to employ at least 20 people in Borlänge, which most annotators read as good news and the model as a neutral fact. Others describe a company, a report or a disclosure ranking. Clear model errors exist as well. The sentence about a net loss that widened to EUR 189,000 from EUR 10,000 has full agreement on negative, and both models call it positive. The baseline misses 14 of the 87 negative sentences, and the weighted model recovers 4 of them, three of which compare two figures, such as a return on investment of 4.1% against 43.8%. Most of the ten sentences that both models miss report bad news without such a comparison, for example traces of a toxic substance on a coast.

For a non-technical reader, both models judge about 84 of 100 sentences the same way as the majority of the annotators. Of 100 negative sentences the normal model finds about 85 and the weighted model about 86. Of 100 sentences that a model flags as negative, about 74 and 73 really carry the negative label. These rates describe my test split, where 12 of 100 sentences are negative. With a smaller share of negative news, and if recall and the false-positive rate stayed the same, the model would raise more false alarms for every negative sentence it finds, so its precision would be lower. For a screening tool that should miss as little as possible I would not choose between the two by these numbers, but compare both at thresholds chosen on validation data.

Limitations and Risks

The test split is small. With 87 negative sentences, one sentence more or less moves the negative recall by 1.1 percentage points, and the differences between the configurations are of

this size. All models are tested on the same split, so the seeds show the variation of the training but not the variation that another split would bring. So the t-tests say nothing about the uncertainty over new training or test samples. The class weights follow a fixed rule, the decision uses no tuned threshold, and I did not check whether the confidence of the model matches its real error rate. My best-epoch rule selects on macro-F1 in both configurations and not on the negative recall that the weighted loss aims at, so it may hide part of the effect of the weighting. The template rule is my own, and the five closest pairs across the splits are still variants of a template, so the results may be optimistic and do not measure entirely new companies or news events. I did not measure how much the remaining template overlap affects the scores.

The labels describe the view of an investor, not of a compliance analyst. A fall of profit is negative for an investor, but adverse media in a due diligence check means allegations such as fraud, corruption or sanctions breaches. The model also does not know which company is being checked, and it gives one label to the whole sentence, even when the sentence names several companies, like the sentence about DNA and Maingate among the error examples. Finally the news is old and regional. 94.8% of the 1,606 years that the sentences mention lie between 2004 and 2011, and many sentences are about Finnish companies.

Ethical and Responsible Use

The main ethical risk I see is the false alarm. In the main run the baseline calls 25 neutral or positive sentences negative, the weighted model 32, and in three of the five seeds the weighted loss raises the number of false alarms. In compliance practice negative news can inform the decision to take on, keep or exit a business relationship, and every alert needs a check whether it concerns the customer and whether it is material (Wolfsberg Group, 2022). An unchecked flag

could therefore lead to an unnecessary restriction for a company, although my experiment does not measure such consequences.

There is also a risk of bias. The model card of the encoder warns that it can make biased predictions and inherit the bias of BERT (Hugging Face, n.d.-b), and I did not evaluate the performance separately for companies from different regions. So the model should only sort and flag news items for an analyst who reads the full article. Before any use I would measure the error rates by region and set the threshold for the negative class together with the compliance team.

Future Improvements

The first step is a stronger evaluation. K-fold cross-validation as in Araci (2019) and more seeds would show whether the small shift of the weighted loss holds across splits. The folds would need a broader grouping than my template rule, for example by company or news event, because the same rule would keep the template links that it misses now. I would first need company and article identifiers, which the released files do not contain. With five seeds the paired t-test has little power, so I would use more seeds and plan their number around a meaningful recall difference and a chosen power. After that I would tune the decision threshold, try a milder class weight and check the calibration. The second step is an encoder with additional pretraining on financial text, such as FinBERT (Araci, 2019), in the same experiment. FinBERT is based on BERT and not on DistilBERT, so to isolate the effect of the financial pretraining I would compare it with the same BERT encoder before this step. Araci (2019) found only a small gain from this further pretraining, so the effect would have to be tested. Finally, adverse media screening needs sentences labeled for compliance relevance, such as allegations of fraud, corruption or sanctions breaches, from more regions and recent years.

References

Araci, D. (2019). *FinBERT: Financial sentiment analysis with pre-trained language models*. arXiv. https://doi.org/10.48550/arXiv.1908.10063

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research, 7*, 1–30. https://jmlr.org/papers/v7/demsar06a.html

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)* (pp. 4171–4186). Association for Computational Linguistics. https://doi.org/10.18653/v1/N19-1423

Elangovan, A., He, J., & Verspoor, K. (2021). Memorization vs. generalization: Quantifying data leakage in NLP performance evaluation. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume* (pp. 1325–1335). Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.eacl-main.113

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. https://www.deeplearningbook.org

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering, 21*(9), 1263–1284. https://doi.org/10.1109/TKDE.2008.239

Hugging Face. (n.d.-a). *DistilBERT* [Transformers documentation, version 5.17.0]. Retrieved October 12, 2026, from https://huggingface.co/docs/transformers/v5.17.0/en/model_doc/distilbert

Hugging Face. (n.d.-b). *distilbert/distilbert-base-uncased* [Model card]. Retrieved October 1, 2026, from https://huggingface.co/distilbert/distilbert-base-uncased

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In *International Conference on Learning Representations*. https://openreview.net/forum?id=Bkg6RiCqY7

Malo, P., Sinha, A., Korhonen, P., Wallenius, J., & Takala, P. (2014). Good debt or bad debt: Detecting semantic orientations in economic texts. *Journal of the Association for Information Science and Technology, 65*(4), 782–796. https://doi.org/10.1002/asi.23062

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter*. arXiv. https://doi.org/10.48550/arXiv.1910.01108

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30*. Curran Associates. https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

Wolfsberg Group. (2022). *The Wolfsberg Group frequently asked questions (FAQs) on negative news screening*. https://db.wolfsberg-group.org/assets/b3652a98-4bc6-4b42-9f05-e378a4adbd95/Negative%20News%20Screening%20FAQs%20(2022).pdf